

Tập 11 - Calico - eBPF

iptables vs eBPF Dataplane trong Calico: $O(n)$ vs $O(1)$

Phần 2 — Calico · #eBPF #iptables #performance #dataplane #O(1)



CALICO

BY TIGERA

Mục tiêu tập này

- Giải thích tại sao iptables không scale với số lượng Pods lớn
- Hiểu eBPF Hash Map $O(1)$ lookup vs iptables $O(n)$
- Bật eBPF dataplane trong Calico (cần kernel 5.3+)
- Xem `tc filter` programs được load vào network interfaces

Prerequisites: Cluster Calico từ Tập 9. Ubuntu 26.04 có kernel 6.x/7.x+ — đủ điều kiện eBPF.

iptables: Thiết kế tuyến tính không scale

1000 Pods → ~10.000 iptables rules

Packet đến:

Check rule 1? No

Check rule 2? No

Check rule 3? No

...

Check rule 10000? Yes → ACCEPT (hoặc DROP)

Complexity: $O(n)$ – 10x Pod = 10x thời gian check

Thêm rule mới:

Phải LOCK toàn bộ iptables table

Rewrite TOÀN BỘ chain (không atomic)

→ Brief window khi rules inconsistent

→ Traffic có thể bị drop trong ms

eBPF: Hash Map O(1)

BPF Hash Map:

Key: {src_ip, dst_ip, dst_port, protocol}

Value: ALLOW/DROP

Packet đến:

Hash lookup $\rightarrow O(1) \rightarrow$ ALLOW hoặc DROP

Không phụ thuộc vào số lượng rules!

1000 Pods hay 100.000 Pods $\rightarrow$ cùng lookup time

Thêm rule mới:

Atomic map update (single pointer swap)

$\rightarrow$ Zero downtime, không traffic drop

$\rightarrow$ BPF programs survive Agent restart
(kernel giữ maps ngay cả khi Agent crash)

So sánh iptables vs eBPF Calico

Tiêu chí	iptables	eBPF
Lookup complexity	O(n)	O(1)
Update method	Lock + rewrite chain	Atomic map update
Traffic during update	Brief disruption	Zero downtime
Conntrack	Linux conntrack	eBPF per-flow state
Kube-proxy required	✓	✗ (Calico thay thế)
Kernel requirement	Any	5.3+ (Ubuntu 26.04: 6.x/7.x+)

Cơ chế nạp BPF vào Card mạng: tc filter vs tcx (Mới)

Trong nhân Linux, chương trình BPF được "treo" vào Traffic Control (TC) subsystem:

- **Cơ chế cũ (< Kernel 6.2): `tc filter` (cls_bpf)**
 - Gói tin đi qua bộ máy phân loại mạng truyền thống của TC.
 - Sử dụng lệnh cổ điển: `tc filter show dev eth0 ingress` để verify.
 - Hạn chế: Có overhead nhỏ của bộ lọc cũ, khó quản lý nhiều programs.
- **Cơ chế mới ($\geq$ Kernel 6.6): `tcx` (BPF Link-based)**
 - Gói tin bypass bộ lọc cũ, BPF program được thực thi trực tiếp trên card.
 - **LƯU Ý:** Vì gắn qua BPF links, lệnh `tc filter show` sẽ hoàn toàn trống trơn!
 - Sử dụng lệnh tối tân: `sudo bpftool net show` để verify.
 - Calico tự động nâng cấp lên `tcx` nếu phát hiện nhân Kernel mới (6.x/7.x+).

Khi nào chọn eBPF mode?

- ✓ Production cluster, nhiều Pods (> 50)
 - ✓ Cần low-latency policy enforcement
 - ✓ Kernel 5.3+ (Ubuntu 26.04 luôn đủ)
 - ✓ Muốn bỏ kube-proxy dependency
- ⚠ Calico eBPF chưa thay thế hoàn toàn Cilium về features
- ⚠ L7 policy: Calico eBPF không có (cần Cilium)

Debug eBPF:

```
sudo bpftool net show           # Xem programs gắn qua tcx (Kernel 6.x/7.x+)
tc filter show dev <interface> ingress # Xem programs gắn qua tc filter (Kernel cũ)
sudo bpftool prog list          # Tất cả BPF programs đang chạy
sudo bpftool map list           # Tất cả BPF maps (routes, conntrack...)
sudo bpftool map dump name cali_v4_routes # Dump bảng định tuyến eBPF
```

Lab Time: Bật eBPF và Verify

Chúng ta sẽ thực hành:

1. **Kiểm tra kernel:** `uname -r` và BPF filesystem trên worker.
2. **Bật eBPF:** Patch kube-proxy + FelixConfiguration để bật eBPF dataplane.
3. **Xem programs:** `bpftool net show / tc filter` để verify BPF programs được load thành công.
4. **So sánh rule count:** iptables rule count vs BPF map size.

👉 Hãy làm theo các bước chi tiết trong file `lab-guide.md`

Tập tiếp theo: Packet flow qua veth pair và conntrack — hành trình đầy đủ của 1 packet qua Calico.